

21st Century Alchemy

Part 4: The unsolved mysteries of generalization



Nicholas J. Cooper



Table of Contents

- The triality of regularization, inductive bias, and generalization
- The quest for a general theory of generalization in deep learning
- Case studies of “alchemical magic”
- Open questions and the future of deep learning

Last Time...

- Digital language processing with RNNs
 - Dual to residual networks
 - Vanishing context problem
- Attention mechanisms
 - “Skip connections” for context
- Self attention and multi-head attention
 - Operationalizing attention
- Transformer blocks
 - Attention is **not** all you need!

This Time...

- What “is” generalization?
 - We don’t know... not really.
 - Best we can do: indirectly influence models → regularization and inductive bias
- How to study generalization?
 - *Many* different theories
 - No “complete” theory of generalization in DNNs...yet
- Alchemical magic
 - Particular situations when we “made gold” — found surprisingly controls over generalization

The Generalization Triality

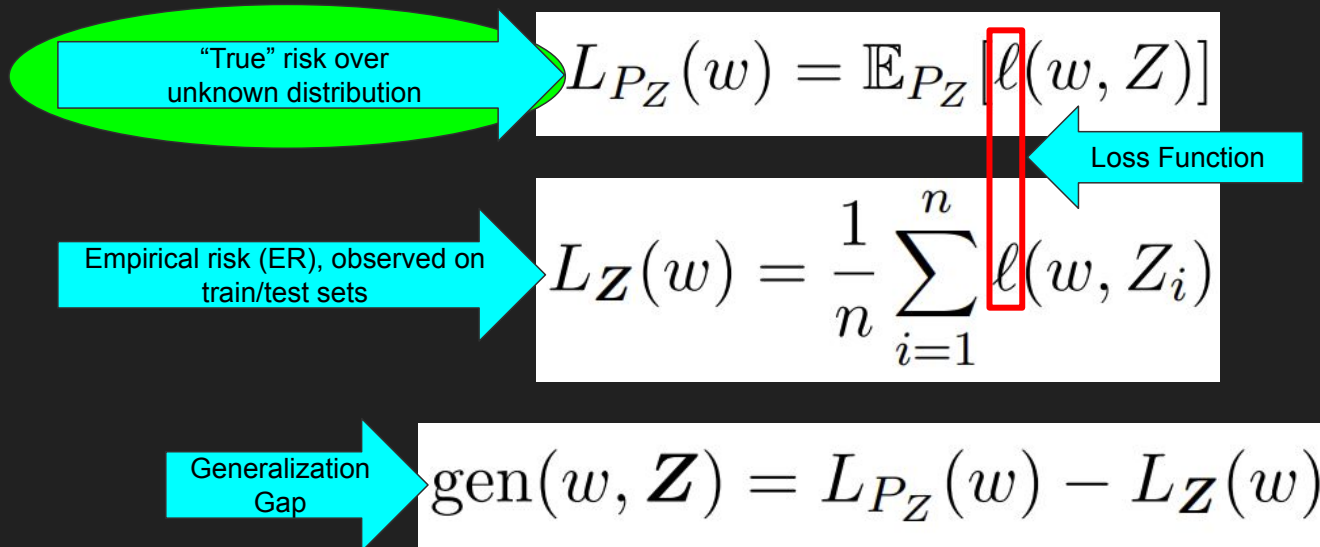
Regularization and inductive bias

What “is” generalization?

- “Pure generalization” has been the ultimate goal of deep neural networks since the very beginning
- Want: trained models to perform well on data samples unseen during training
- Problem: very tricky to actually define this concept of generalization
- Induce generalization indirectly via two complementary techniques:
 - Regularization
 - Inductive bias

Generalization is *not* properly definable

- Most “definitions” of generalization rely on a mysterious “true probability distribution” from which unseen samples are drawn



Equations taken from: *Generalization Bounds: Perspectives from Information Theory and PAC-Bayes*, Hellström et. al., Foundations and Trends in Machine Learning, vol. 18. 2025

The Problem of “True Distributions”

- By definition, we do *not* know much, if anything, about the true distribution used to define generalization
- A straightforward argument:
 - Suppose we did know how to compute loss on samples from the true distribution
 - We must therefore know something about the labels/desired outputs on these samples
 - These samples would then belong to a validation or test set!
- This is a verbose way of saying “we don’t know what we don’t know”
- Poses *major* problems for studying generalization *directly*

Example

- Consider this ridiculous example
- Many might agree that 9 is the “correct” solution
- Yet, without access to the “true” distribution, we *cannot know*
- What if we’re training a model to predict adversarially constructed integer sequences?

“Dataset” →

Find the next number of the sequence

1, 3, 5, 7, ?

Correct solution ?!?

217341

because when

$f(x) = \frac{18111}{2}x^4 - 90555x^3 + \frac{633885}{2}x^2 - 452773x + 217331$

$f(1)=1$
 $f(2)=3$ much solution
 $f(3)=5$ wow very logic
 $f(4)=7$
 $f(5)=217341$ such function
many maths
wow



Meme credit: <https://stats.stackexchange.com/questions/128616/whats-a-real-world-example-of-overfitting>

Example

- Surely 9 is a “simpler” answer, and therefore correct?
- This thinking is known as a *simplicity bias*
- A popular form of *regularization*
 - An assertion of “correctness” enforced on the unknown “true distribution”
- Used frequently in deep learning to improve generalization *indirectly*
- Yet, recent research challenges the effectiveness of simplicity biases
 - See e.g., Teney et. al., CVPR 2025

Find the next number of the sequence

1, 3, 5, 7, ?

Correct solution **217341** **?!?**

because when

$$f(x) = \frac{18111}{2}x^4 - 90555x^3 + \frac{633885}{2}x^2 - 452773x + 217331$$

$f(1)=1$
 $f(2)=3$ much solution
 $f(3)=5$ wow very logic
 $f(4)=7$
 $f(5)=217341$ such function
many maths
wow



Meme credit: <https://stats.stackexchange.com/questions/128616/whats-a-real-world-example-of-overfitting>

But wait, the problem gets worse...

- Modern-day deep neural networks are so expressive, they can interpolate *any* training dataset
 - Meaning, they always achieve ~perfect performance on the known, labeled training samples

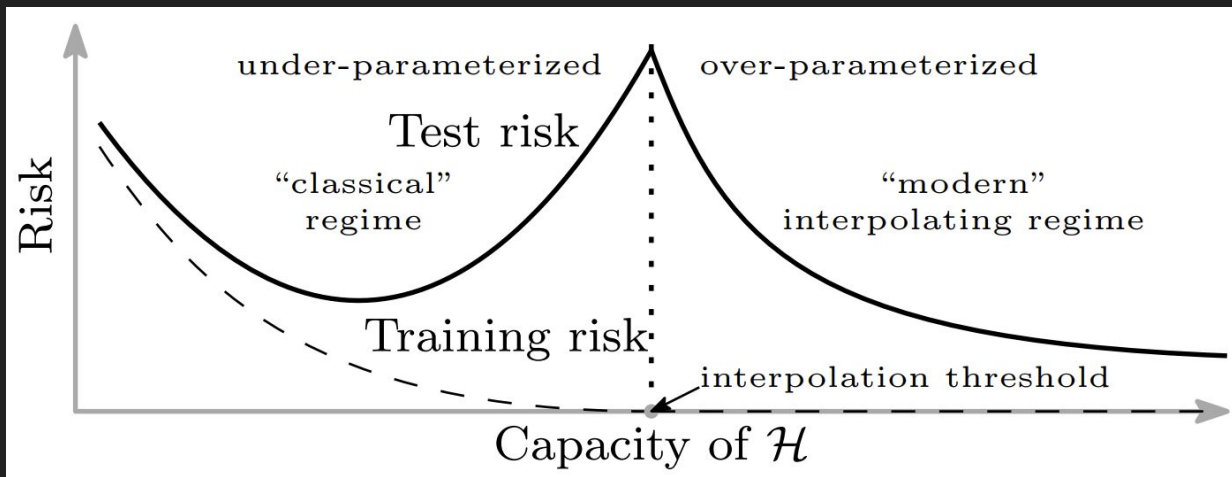


Figure taken from: *Fit Without Fear*, Belkin, 2021

How to select the “correct” solution?

- Today, the question “which models generalize?” is effectively the question “which solution (of *many*) is correct?”
 - Technically, we say that the solution set has *positive dimension* in parameter space
- This is, of course, an ill-posed problem

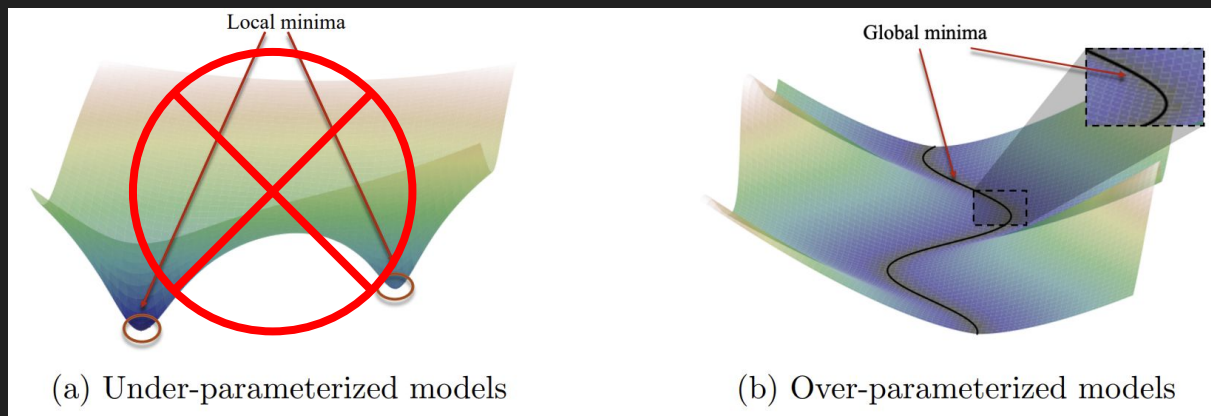
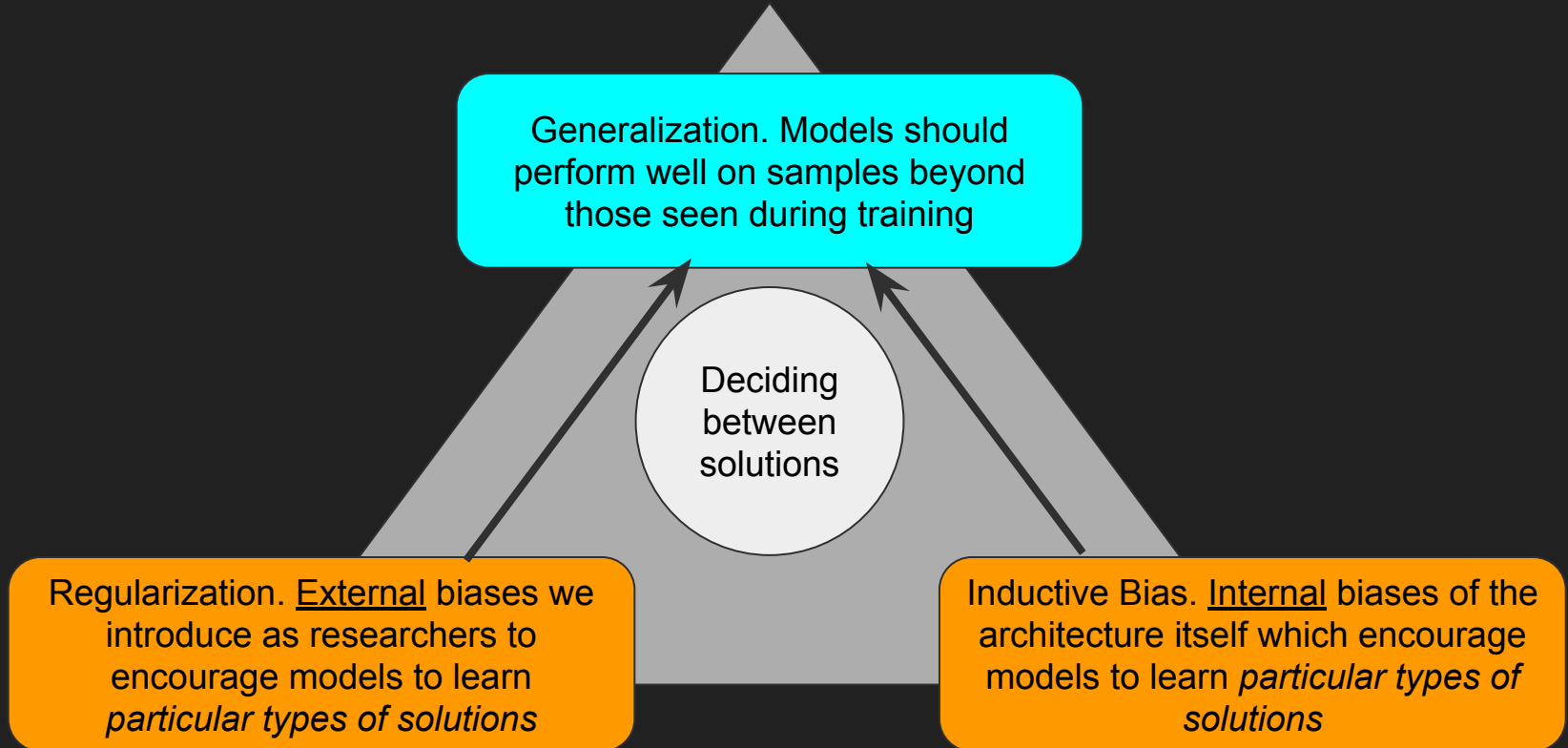


Figure taken from: *Fit Without Fear*, Belkin, 2021

The best we can do: target generalization indirectly

- There are two primary ways to influencing the generalization ability of deep learning models
- *Regularization* refers to techniques which enforce *external constraints* on the types of solutions learned by the model
 - Common examples: simplicity biases, hyperparameter choices, training recipes
- *Inductive bias* refers to the *implicit constraints* the model architecture enforces on the types of solutions it is likely to learn
 - Common examples: CNN translation equivariance, Transformer permutation equivariance

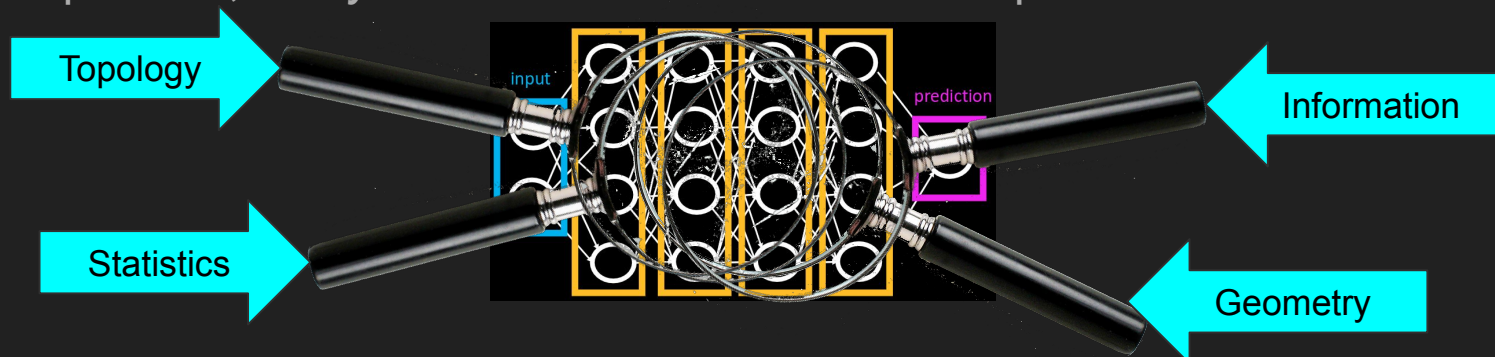
Triality: Generalization, Regularization, Inductive Bias



Theories of Generalization for Deep Learning

How might we study generalization formally?

- This question is a primary goal of machine learning theory
 - Can we develop tools for building better architectures? Regularizers?
 - Can we understand why and/or how models learn to generalize?
- *Very* difficult due to the ill-defined nature of generalization itself
- Despite this, *many* different theories have developed



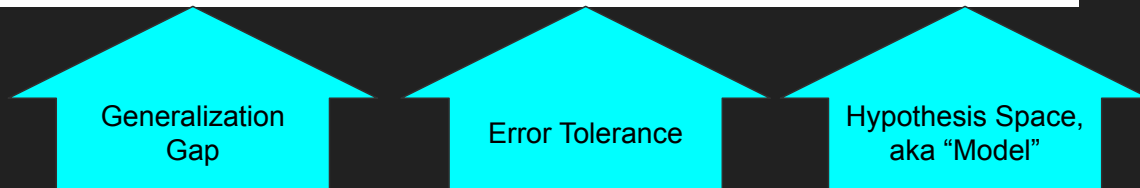
Picture credits: <https://home.cs.colorado.edu/~DrG/Courses/2026-Spring-NN/03-Fully-Connected-NN.pdf>

Theory #1: Statistical Learning Theory

- One of the oldest approaches to the formal study of generalization (~1970s)
- Key idea: just work with the generalization gap directly!
- Key tool: PAC (probably approximately correct) bounds

with probability at least $1 - \delta$

$$|L_Z(w) - L_{P_Z}(w)| \leq \epsilon \quad \text{for all } w \in \mathcal{W}$$



Equations taken from: *Generalization Bounds: Perspectives from Information Theory and PAC-Bayes*, Hellström et. al., Foundations and Trends in Machine Learning, vol. 18. 2025

Theory #1: Statistical Learning Theory

- Prototypical results: “complexity” bounds on the generalization gap
- The below bound is due to Shalev-Shwartz and Ben-David, 2014
- Utilizes the **VC dimension** — a measure of model expressivity

The diagram illustrates the relationship between the Generalization Gap, the VC dimension, and the complexity bound. A light blue arrow labeled "Generalization Gap" points to the left side of the equation. A light blue arrow labeled "Complexity" points to the right side of the equation. The equation itself is $|L_Z(w) - L_{P_Z}(w)| \leq \sqrt{C \frac{d_{VC} + \log \frac{1}{\delta}}{n}}$. The term d_{VC} is highlighted with a red box, and the entire right-hand side of the inequality is enclosed in a red box.

$$|L_Z(w) - L_{P_Z}(w)| \leq \sqrt{C \frac{d_{VC} + \log \frac{1}{\delta}}{n}}$$

Theory #1: Statistical Learning Theory

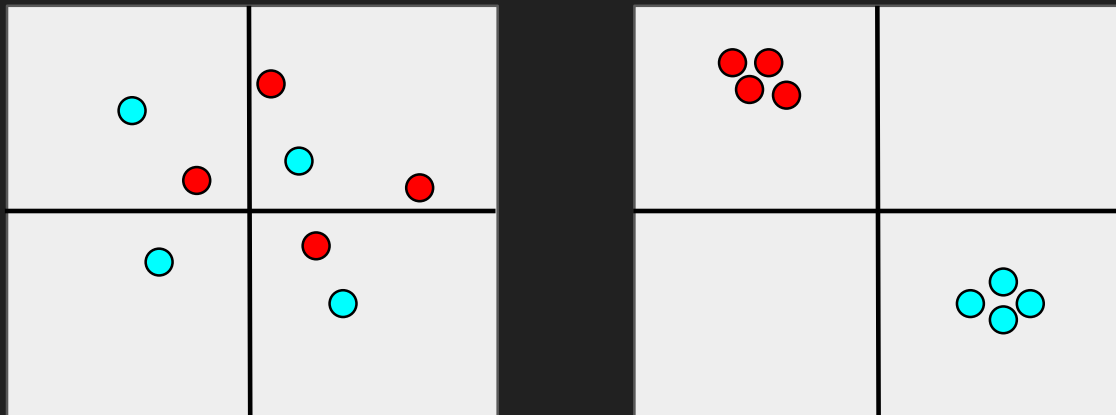
- **The catch:** many of the bounds provable w/ SLT techniques do *not* work
 - At least for practically viable deep neural networks
- When put to the test on real-world neural networks, most generalization gap bounds are *negatively correlated* with performance!

		batch size	dropout	learning rate	depth	optimizer	weight decay	width	overall τ	Ψ
Corr	vc dim 19	0.000	0.000	0.000	-0.909	0.000	0.000	-0.171	-0.251	-0.154
	# params 20	0.000	0.000	0.000	-0.909	0.000	0.000	-0.171	-0.175	-0.154
	$1/\gamma$ (22)	0.312	-0.593	0.234	0.758	0.223	-0.211	0.125	0.124	0.121
	entropy 23	0.346	-0.529	0.251	0.632	0.220	-0.157	0.104	0.148	0.124
	cross-entropy 21	0.440	-0.402	0.140	0.390	0.149	0.232	0.080	0.149	0.147
	oracle 0.02	0.380	0.657	0.536	0.717	0.374	0.388	0.360	0.714	0.487
	oracle 0.05	0.172	0.375	0.305	0.384	0.165	0.184	0.204	0.438	0.256
	canonical ordering	0.652	0.969	0.733	0.909	-0.055	0.735	0.171	N/A	N/A

Figure taken from: *Fantastic Generalization Measures and Where to Find Them*, Jiang et. al., ICLR, 2020

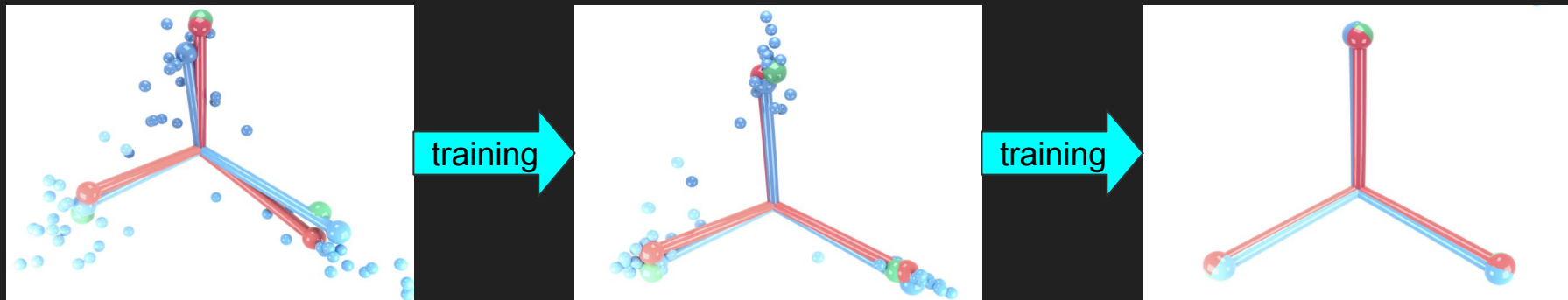
Theory #2: Latent Representation Geometry

- Perhaps a direct upper bound on generalization error is too much to hope for
- Can we still glean some insights on how DNNs achieve generalization?
- Key idea: investigate the geometric structure of internal DNN features
- Key tool: dot product similarity and *intrinsic dimension*



Theory #2: Latent Representation Geometry

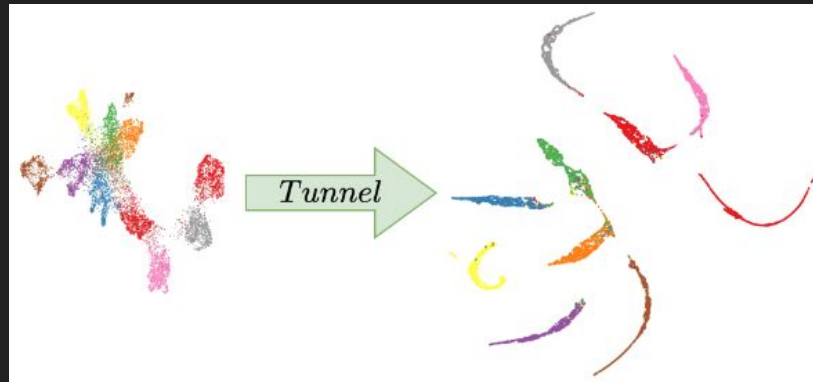
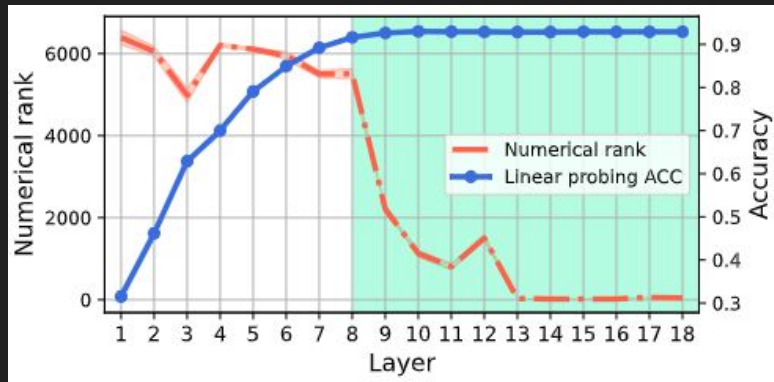
- A prototypical result: *deep neural collapse*
- Discovered in 2020 by Papayan, Han, and Donoho
- DNNs consistently collapse final latent representations to simplex frames
- Happens *automatically* — without being trained explicitly for such collapse



Figures taken from: *Prevalence of Neural Collapse during the terminal phase of deep learning training*, Papayan et. al., PNAS, 2020

Theory #2: Latent Representation Geometry

- A prototypical result: *the tunnel effect*
- Popularized in '23, but this line of work started in '19 by Ansuini et. al.
- DNNs *extract* then *compress* features across their hidden layers
- Happens *automatically* — without being trained explicitly for such behavior



Figures taken from: *The Tunnel Effect: Building Data Representations in Deep Neural Networks*, Masarczyk et. al., NeurIPS, 2020

Theory #2: Latent Representation Geometry

- **The catch:** analyzing the geometric structure of representations tells us about *what* DNNs do to accomplish generalization, not *how* or *why*
- Useful for studying specific instances of *trained architectures*
- Somewhat useful for developing new regularization techniques
 - Via geometrically informed loss terms
- Unhelpful for guiding development of *new architectures*

Theory #3: Neuro-Algebraic Geometry

- If the geometric structure of latent features is useful, what about the geometric structure of *architectures* themselves?
- Key idea: study the geometry of function spaces defined by architectures
- Key tool: algebraic varieties allow us to build* **neuro-manifolds** from **parameterization maps**

$$\varphi: \mathcal{W} \rightarrow \mathcal{M}, w \mapsto f_w.$$

$$f(w) = f_w$$

$$f_w(x) = y$$

Algebraic geometry

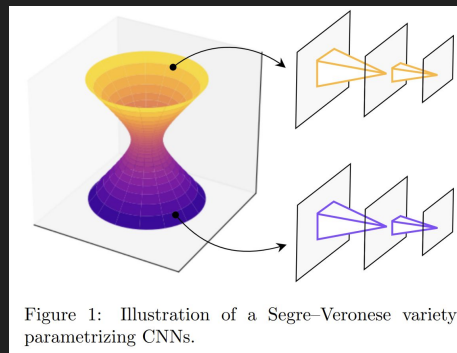


Figure 1: Illustration of a Segre-Veronese variety parametrizing CNNs.

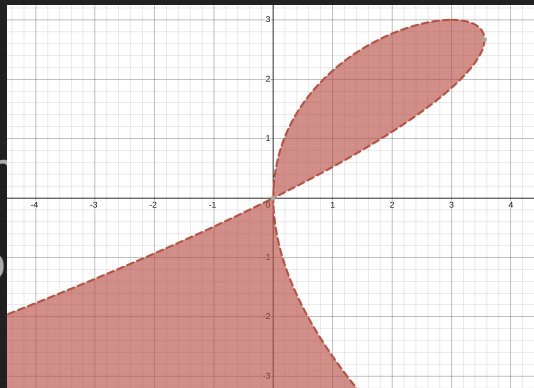
* sometimes, and with immense difficulty

Equation taken from: *Algebra Unveils Deep Learning: An Invitation to Neuroalgebraic Geometry*. Marchetti et. al., ICML 2025.

Figure taken from: *On the Geometry and Optimization of Polynomial Convolutional Networks*. Shahverdi et. al., 2024.

Theory #3: Neuro-Algebraic Geometry

- A prototypical result: MLPs with quadratic activation
- Using the fact that the **semi-algebraic sets** are closed and bounded, we can construct the neuro-manifold for such MLPs
- Neuro-manifolds allow us to analyze *architectures*, not just specific instances:
 - Formally define the *dimension* of an architecture as the manifold dimension of its function space. This is a **well-defined version** of “model expressivity”!
 - Study the singular points of architectures — points in parameter space that are likely to be found during training. This is **well-defined version** of “inductive bias”!



Theory #3: Neuro-Algebraic Geometry

- **The catch:** building neuro-manifolds is not always possible, and even when it is, it can be extremely difficult depending on the architecture.
- We cannot construct neuro-manifolds for ReLU activated networks.
 - **Tropical geometry** provides tools for ReLU networks, but has its own limitations
- Even when we can produce a neuro-manifold, it is not yet clear how to translate that achievement into practical gains.
 - Can we build algorithms to decide which architecture to use?
 - Compare architectures? Improve regularization techniques?
- This field is very new (~2019). Much work required to establish practical value

Theory #4: Geometric (and Categorical) Deep Learning

- **Recall:** 2-d convolution is equivariant to grid translations; self-attention is equivariant to token permutations.
 - These functions comprise two of the most influential types of deep neural networks.
 - Can we study general equivariant architectures?
- Key idea: write down the type(s) of equivariance we want *first*, then build the right type of neural network *second*. Produce **equivariant layers**.
- Key tools: symmetry groups, grids, graphs.

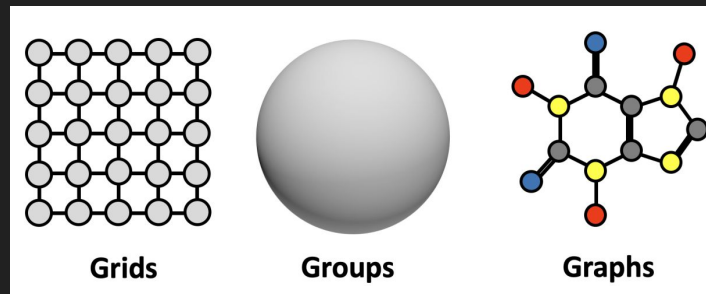
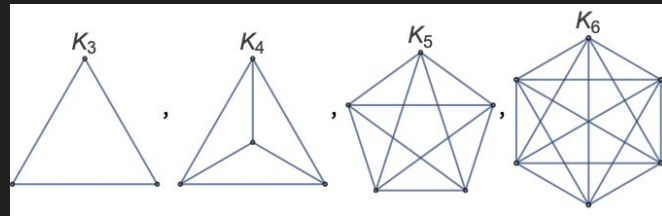


Figure taken from: *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*. Bronstein et. al., 2021.

Theory #4: Geometric (and Categorical) Deep Learning

- Many different networks can be understood by their equivariance properties
 - Convolutional layers (translation on grids)
 - Self-attention layers (permutation on *complete* graphs)
 - Graph neural networks (permutation on graphs)
 - ...



- Geometric deep learning demonstrates how “top-down design” is possible

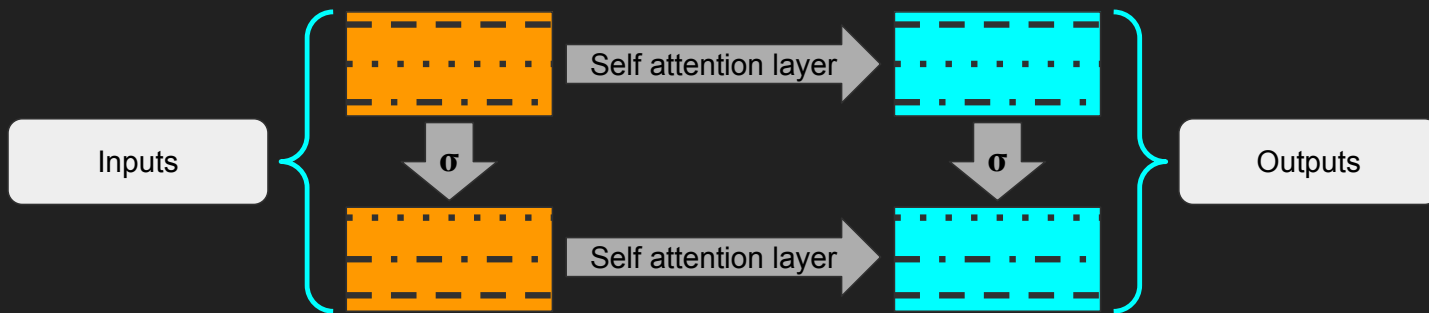


Figure credit: <https://reference.wolfram.com/language/ref/CompleteGraph.html>

Theory #4: Geometric (and Categorical) Deep Learning

- **The catch:** analyzing equivariance properties can highlight *which* models generalize, but it doesn't explain *how* models generalize.
- It is common practice to *augment* image data with random translations
 - 2-d convolutional layers are translation equivariant.
 - This Improves CNN performance *significantly*.
- Not all relevant data transformations are “easy” to model w/ group actions.
 - **Categorical deep learning** extends “top down design” to more sophisticated transformations.
- Similar to neuro-algebraic geometry, constructing solutions to equivariance constraints is not always easy or possible.

Theory #5: Topological Deep Learning

- **Recall:** the topological features of computational graphs are important
 - Residual blocks — parallel paths between input and output
 - RNNs — parallel paths between parameters and output
 - Self-attention — parallel paths between tokens and output
- Key idea: can we describe the topological structure of deep neural networks?
- Key tool: **copresheafs** — a way to attach linear transformations to graphs

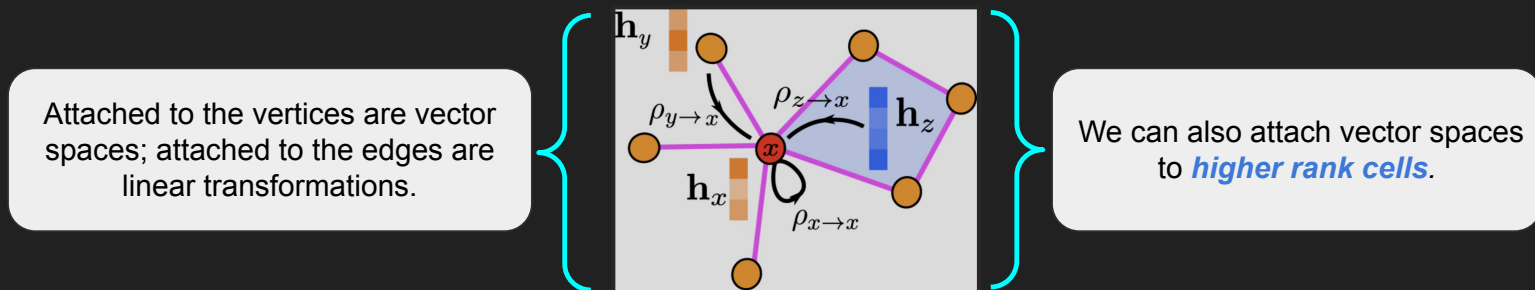


Figure taken from: *Copresheaf Topological Neural Networks: A Generalized Deep Learning Framework*. Hajij et. al., NeurIPS 2025.

Theory #5: Topological Deep Learning

- Copresheafs can describe many types of neural networks
 - CNNs, GNNs, Transformers, and more.
- Copresheafs can be used to build new, better architectures!
 - Improves performance on classification of molecular graphs.
- Overview of construction:
 - Inject extra linear transformations into a graph neural network
 - “Don’t just send the message”, transform it first
 - These extra transformations increase model expressivity
- Intuition: data controls the structure of the network

Model	Accuracy
GCN	0.674 ± 0.014
<u>CopresheafGCN</u>	0.721 ± 0.035
GraphSAGE	0.689 ± 0.022
<u>CopresheafSage</u>	0.732 ± 0.029
GIN	0.700 ± 0.039
<u>CopresheafGIN</u>	0.724 ± 0.021

Figure taken from: *Copresheaf Topological Neural Networks: A Generalized Deep Learning Framework*. Hajij et. al., NeurIPS 2025.

Theory #5: Topological Deep Learning

- **The catch:** copresheafs are highly general constructions that help *build new models*, but the framework doesn't explain *how* they generalize.
- Details of the linear transformations between vertices (and higher rank cells) are hidden away.

A word from our “sponsors”...

Think neural networks are becoming more “complex”?

Not sure how to formalize this intuition?

Check out our recent paper!
“On the Architectural Complexity of
Neural Networks”

Available ***for free*** at:

<https://arxiv.org/pdf/2605.04325>

Theory #6: Combinatorial Deep Learning

- **Recall:** deep neural networks are implemented as computational graphs derived from (increasingly intricate) compositions of tensor operations.
- We formalize architectural complexity by encoding computational graphs as high rank cell hierarchies
- This provides analysis tools, and ways of building new, **efficient** types of computational structures

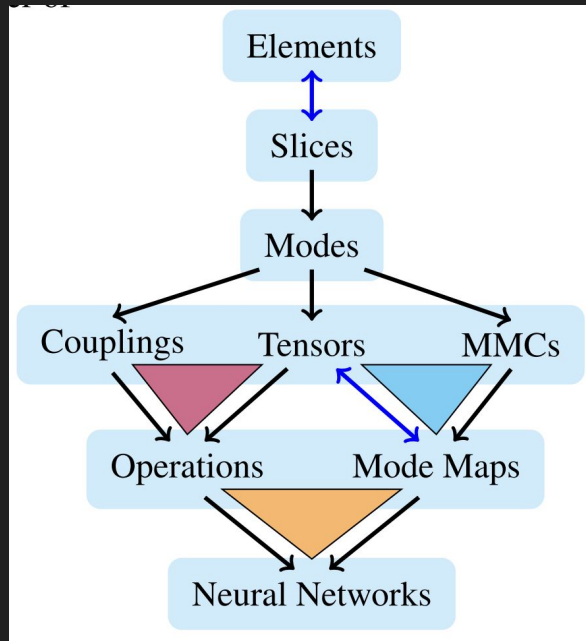


Figure taken from: *On the Architectural Complexity of Neural Networks*. Cooper et. al., Preprint 2026.

Theory #6: Combinatorial Deep Learning

- Just like copresheaf networks, cell hierarchies describe many architectures
- Keeping track of cell counts gives measures of architectural complexity

	Fundamental Architectures				Post-Transformer Architectures			
	FCNN	CNN	ResNet	Transformer	Poly-Net	MO-Net	ViM	TT-Net
Year	1986	1998, 2012	2016	2017	2020	2024	2024	2025
$\mathcal{C}_{op}$	1	1 → 3	3	3	4	4	27	5
$\mathcal{C}_T$	2	2 → 6	6	7	9	9	45	11
$\mathcal{C}_\alpha$	2	2	2 → 3	3	3	4	3	3
$\mathcal{C}_O$	3 → 6	6	6	4	6	4	4	6
$\mathcal{C}_A$	2	2	2	2	2	2	2 → 3	3

Architectures from the first explosion

Architectures from the second explosion

- Studying computational graph complexity helps *build new architectures*
 - Some of which exhibit high parameter efficiency!

Figure taken from: *On the Architectural Complexity of Neural Networks*. Cooper et. al., Preprint 2026.

Theory #6: Combinatorial Deep Learning

- **The catch:** cell hierarchies are useful for understanding architectural complexity, but they don't explain *why* higher complexity models do better.
- Describing “what” doesn't provide insight into “how”
 - Similar to the trade-offs of copresheaf networks from topological deep learning

Alchemical Magic

Case Studies

- The super convergence phenomena of the OneCycle learning rate scheduler
 - Train models in a fraction of the time required by other schedulers
 - A study of **regularization** induced generalization
- The surprising power of *random* convolutional layers
 - Learning without updating any convolutional parameters
 - A study of **inductive bias** induced generalization
- Training backbones without ground truth gradients
 - Teach the majority of layers without any direct task supervision
 - A study of **regularization** induced generalization

Case Study #1: OneCycle Learning Rate Schedule

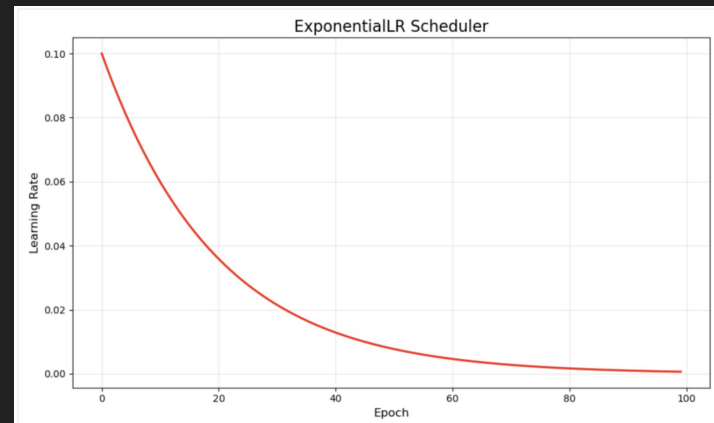
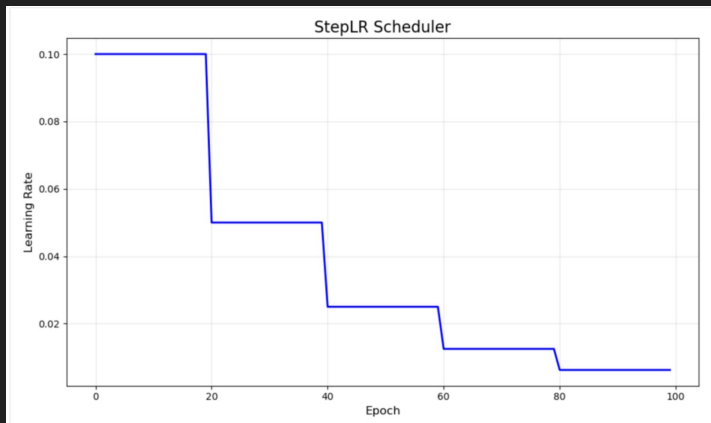
- **Recall:** the learning rate (aka step size) is a critical hyperparameter for training DNNs
- Key idea: modify learning rate across training samples
 - $\alpha = f(t)$, t is the training iteration
 - Origins traceable back to early 2010s
 - Related to adjustable learning rate optimizers, e.g., [Adam](#) (Kingma and Ba, 2014)
- Intuition:
 - Geometry of the loss landscape changes as training progresses
 - Constant learning rate will never be optimal at all iterations

$$W_{new}^{[1]} = W_{old}^{[1]} - \boxed{\alpha} \frac{dJ}{dW^{[1]}}$$

Picture credits: <https://medium.com/data-science/neural-networks-backpropagation-by-dr-lihi-gur-arie-27be67d8fdce>

Case Study #1: OneCycle Learning Rate Schedule

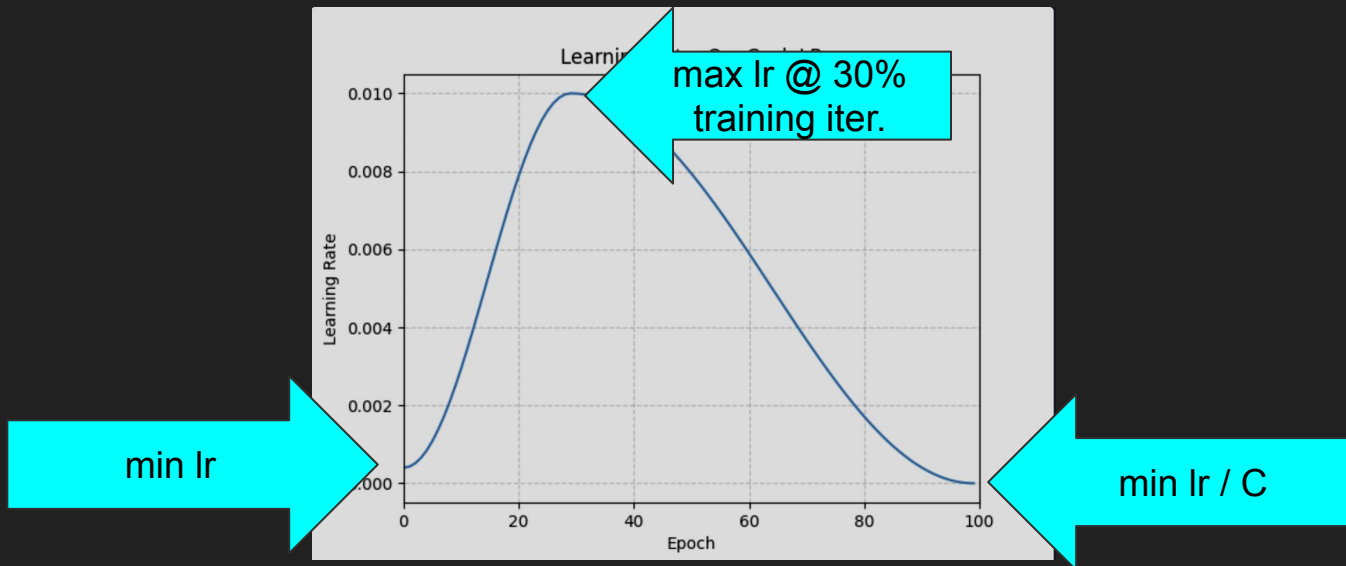
- Many different types of learning rate schedules exist
 - StepLR
 - ExponentialLR
 - CosineLR
 - ...



Picture credits: <https://machinelearningmastery.com/a-gentle-introduction-to-learning-rate-schedulers/>

Case Study #1: OneCycle Learning Rate Schedule

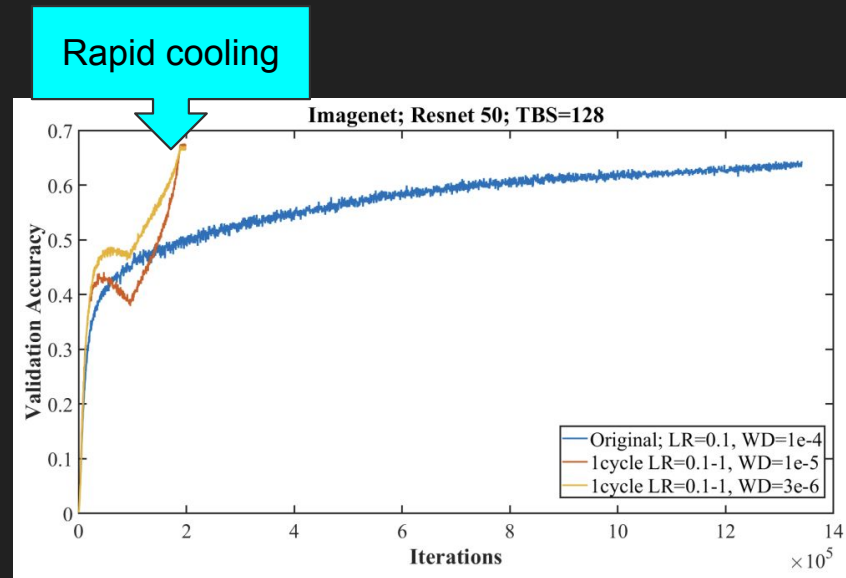
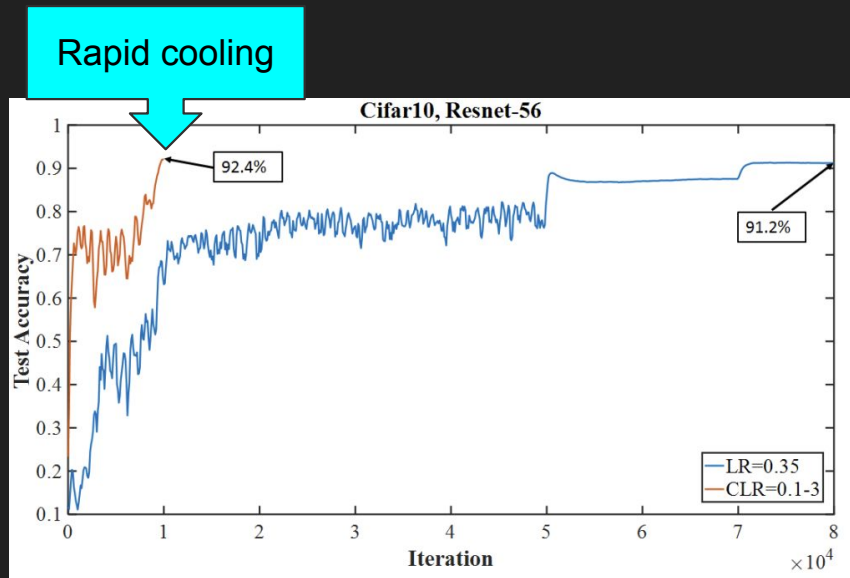
- In 2017, Leslie Smith and Nicolay Topin introduced the *one cycle schedule*
 - “Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates”



Picture credits: https://docs.pytorch.org/docs/2.12/generated/torch.optim.lr_scheduler.OneCycleLR.html

Case Study #1: OneCycle Learning Rate Schedule

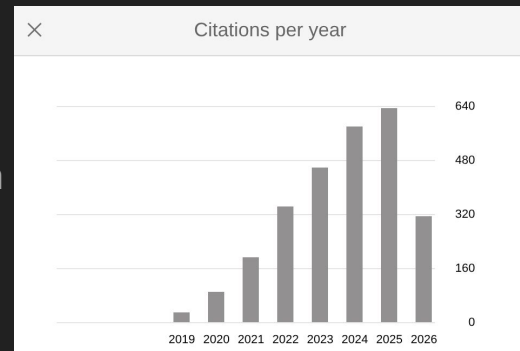
- This scheduler is *extremely effective* at training image classification models!
 - Convergence in an order of magnitude fewer training iterations



Picture credits: Smith and Topin, *Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates*, 2017.

Case Study #1: OneCycle Learning Rate Schedule

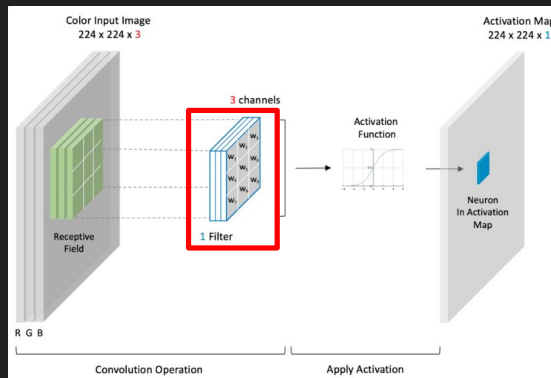
- Response:
 - Super Convergence paper gets *rejected* from ICLR 2018
 - OneCycle scheduler gets an official PyTorch implementation
 - Gone on to influence much research — over 2,500 citations
- Takeaways:
 - The right learning rate at the right time is powerful regularization
 - *A very particular* schedule achieves remarkable results
 - Provides insight into the structure of loss landscapes
- Notes:
 - I use OneCycle *exclusively* for image classification work
 - Facilitates rapid experimentation and prototyping



Picture credits: https://scholar.google.com/scholar?cites=2219536765070582068&as_sdt=5.46&scioldt=0.46&hl=en

Case Study #2: Very Deep Random CNNs

- **Recall:** CNNs use convolutional layers with many different *filters*
- Key question: what drives the generalization ability of CNNs?
 - Their ability to learn, or their architectural structure itself?
- Key idea: evaluate CNNs with *random filters*
 - No training for any convolutional layer — isolate effectiveness of the architecture in isolation



Picture credit: <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>

Case Study #2: Very Deep Random CNNs

- In practice, **batch norm layers** are used to build CNNs
 - These layers perform learned affine transformations

$$BN(\mathbf{X}) = \gamma \frac{\mathbf{X} - \mu(\mathbf{X})}{\sigma(\mathbf{X})} + \beta$$

Diagram illustrating the Batch Normalization (BN) formula:

- $\mu(\mathbf{X})$ is labeled "Batch mean" (indicated by a red arrow pointing to the term).
- $\sigma(\mathbf{X})$ is labeled "Batch standard deviation" (indicated by a red arrow pointing to the term).
- γ and β are highlighted with red boxes, representing learned parameters.

- The learned parameters γ and β account for only a tiny fraction of a CNN's total parameter count
 - For example, ~35 thousand out of 25 *million* parameters for representative CNN architectures
- In 2019, Mudrarkarta et. al investigated what happens when *only* γ and β receive gradient updates

Case Study #2: Very Deep Random CNNs

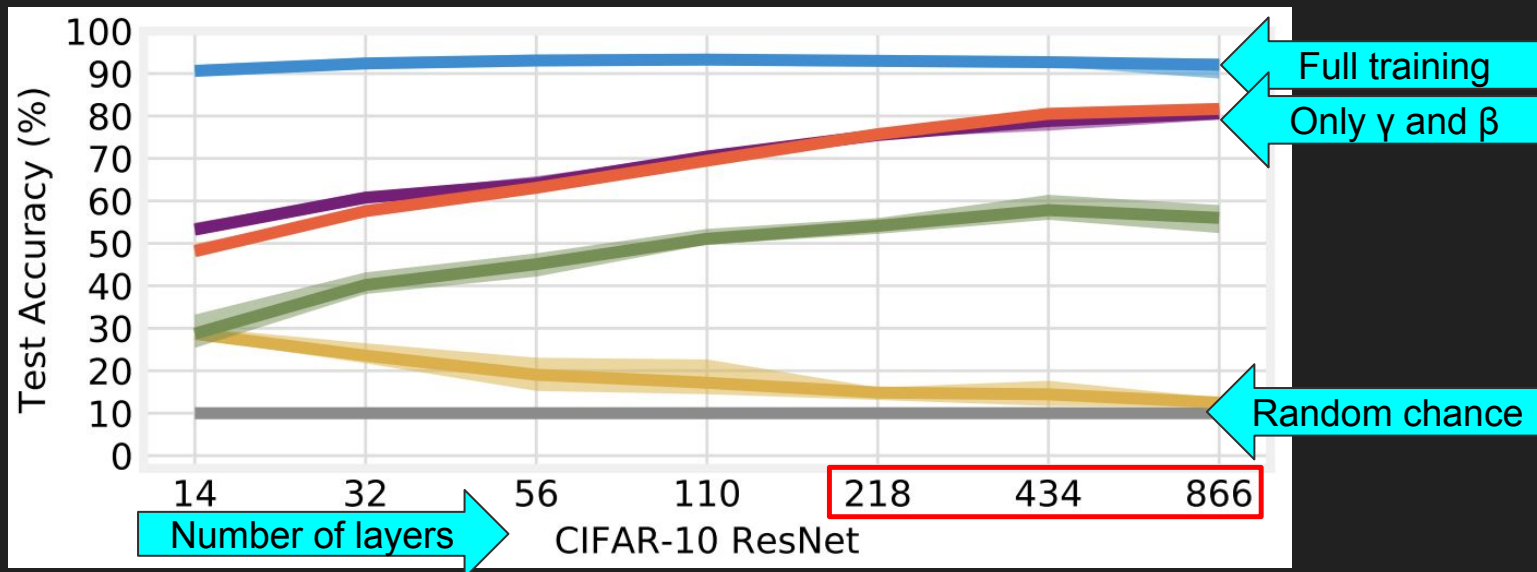
- First hints at something surprising:
 - Training just γ and β produces models that *still generalize pretty well*
 - Original experiments focused on **transfer learning**

Fine-tuned params.	Flowers		Cars		Aircraft	
	Acc.	#params	Acc.	#params	Acc.	#params
Last layer	84.5	208K	55	402K	45.9	205K
S/B + last layer	90.4	244K	81	437K	70.7	241K
S/B only (random last)	79.5	36K	33	36K	52.3	36K

- In 2021, Frankle et. al. pushed this idea much further
 - It is possible to build CNNs that generalize without ever training the filters?

Case Study #2: Very Deep Random CNNs

- Amazing finding: CNNs can indeed generalize without any learned filters!
 - Caveat: they must be *much deeper* than traditional architectures

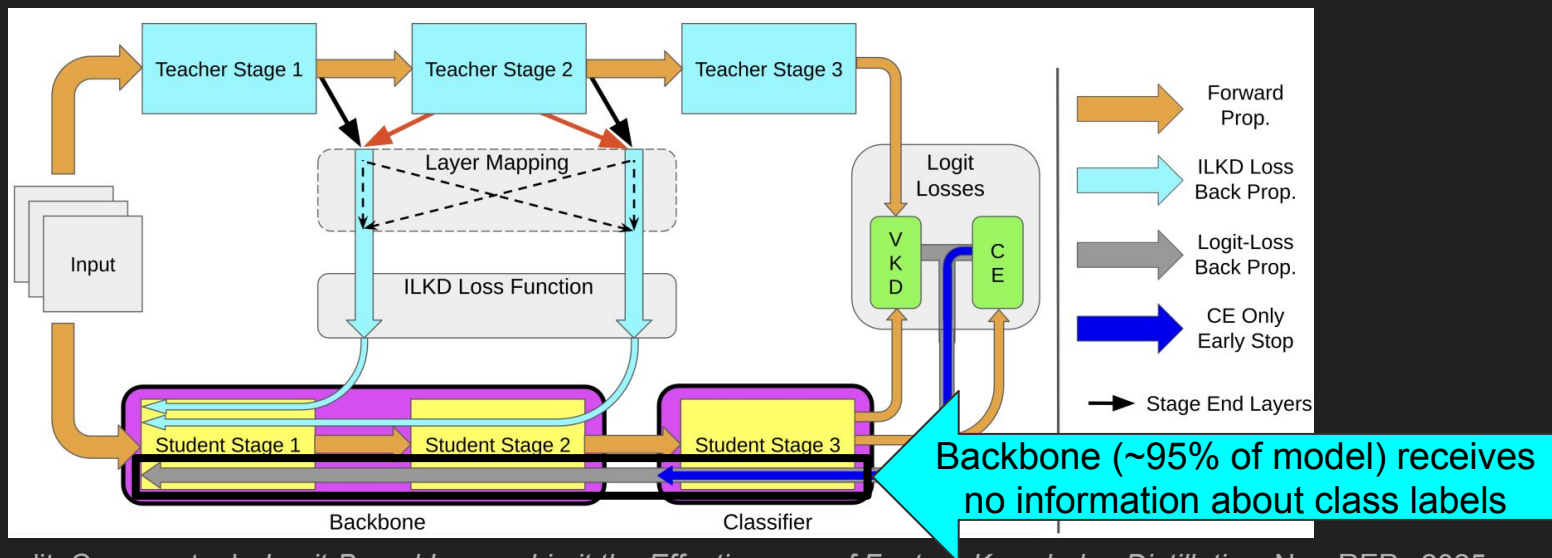


Case Study #2: Very Deep Random CNNs

- These surprising findings inspired much research
 - Related to the [lottery ticket hypothesis](#) (also Frankle et. al.)
 - Very large DNNs contain “winning tickets” right from the very beginning (before training)
 - LTH is in turn related to neuro-algebraic geometry via. the notion of [subnetworks](#)
 - In 2024, Cao and Wu demonstrated ([TOBIAS](#)) that CNNs can detect objects without *any* training, not even training of batch norm layers.
- Key takeaways:
 - CNN architectures exhibit remarkable *inductive biases*
 - Translation equivariance of convolutional layers is perhaps not the whole picture
 - There are connections between model depth and the number of parameters that require training

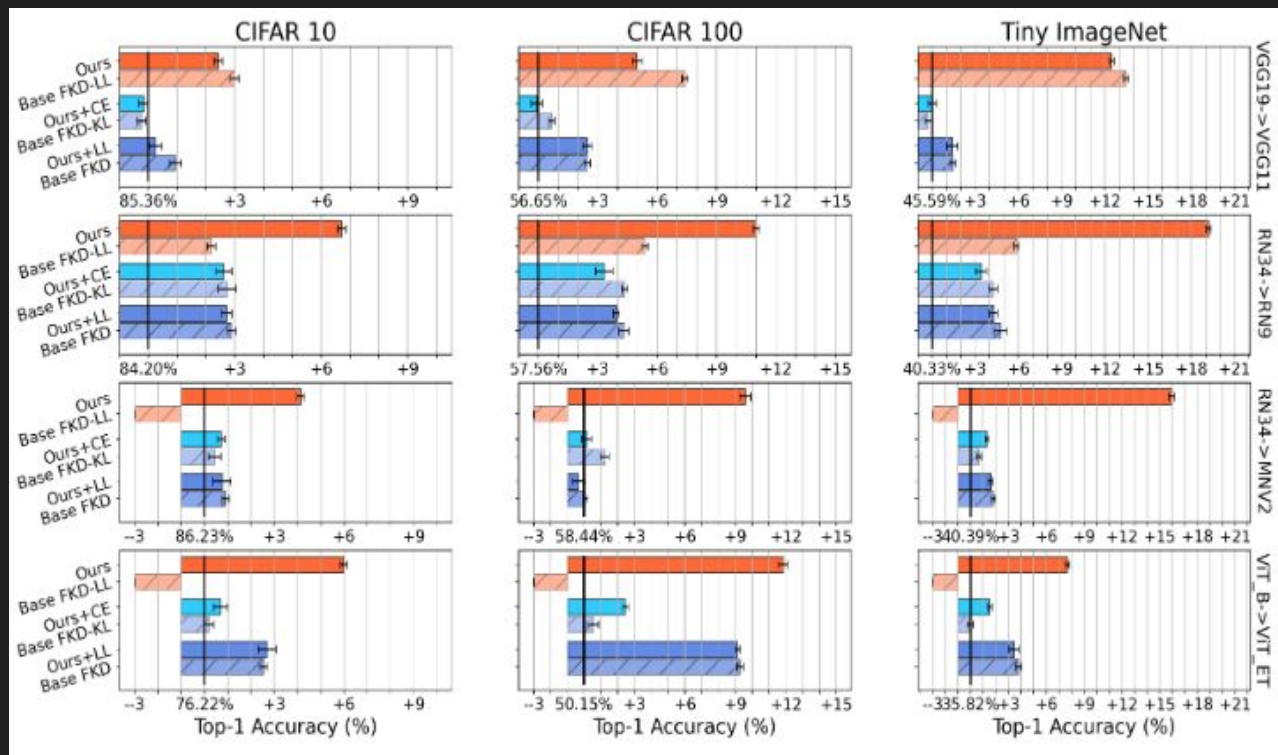
Case Study #3: Training Backbones w/o Ground Truth

- In a [previous paper](#), we showed how to train models without using any information from the ground truth labels!
 - Surprisingly, this works very well in particular cases, but is highly sensitive to impl. details



Case Study #3: Training Backbones w/o Ground Truth

This actually works!*



* highly sensitive to layer selection

The Future of Deep Learning

Course Summary

- The Perceptron started everything in 1957
 - Linear decision boundary, simple parameter update rule
- Deep learning was not “deep” until 1986
 - Non-linear decision boundaries, complicated parameter update rule
- Fully connected networks lack useful inductive biases → CNNs in 1989
 - Translation equivariance, pure generalization, and the 1st architecture explosion
- Deep learning was not “very deep” until 2016
 - Residual networks, skip connections, and much deeper models
- Linear->activation->linear->activation... until 2017
 - Self attention and transformers revolutionized architecture design

Course Summary

- What exactly “is” generalization — it’s hard to say; term is ill-defined
 - Regularization and inductive bias are the primary tools we have for inducing generalization
- The zoo of theories for deep learning
 - Focus on generalization directly → statistical learning theory
 - Focus on equivariance properties → geometric/categorical deep learning
 - Focus on architecture structure → topological/combinatorial deep learning
- Evidence of (regularization/inductive bias) → generalization
 - Very particular learning rate schedules enable “super convergence”
 - CNN’s inductive bias enables generalization without training

The Scaling Problem

- It is well known by now that DNNs face exponential computational problems
- More additional training compute is required for comparatively lower returns

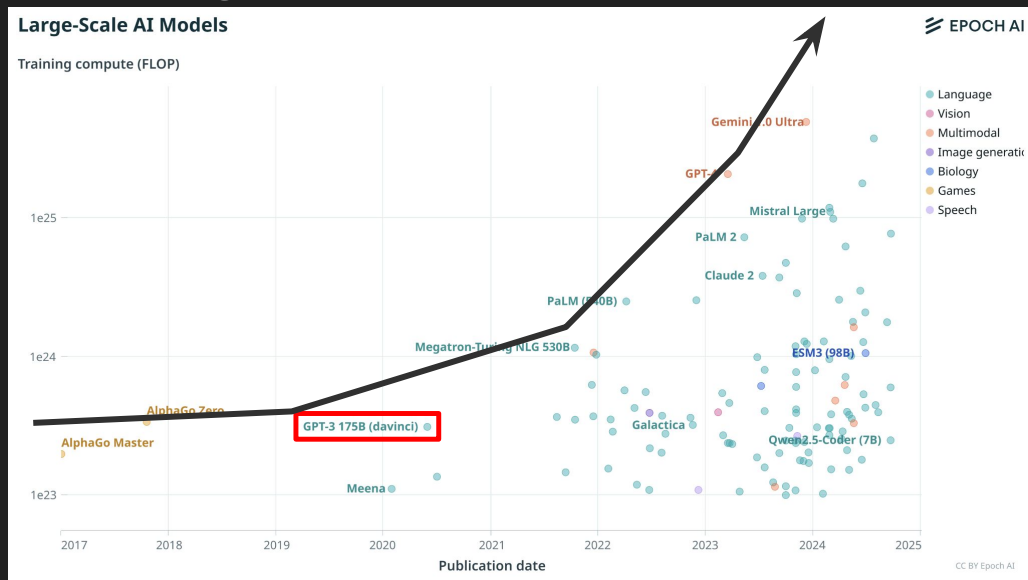


Figure credit: [Wikipedia](#), [Epoch AI](#)

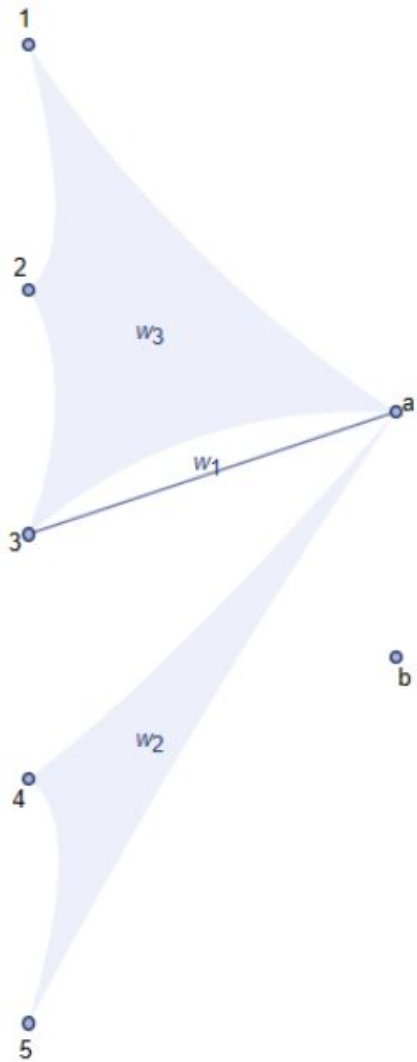
Are we stuck in an architectural local minima?

- Attention was developed ~10 years ago
- Transformers emerged ~8 years ago
- Since then, the status quo has been to build larger and larger transformers
- Many variants and engineering tricks:
 - Linear attention, flash attention, etc...
 - Knowledge distillation, quantization, etc...
- Many new combinations of layers...
- But no architecture breakthroughs
 - Are attention-based models the best we can do?

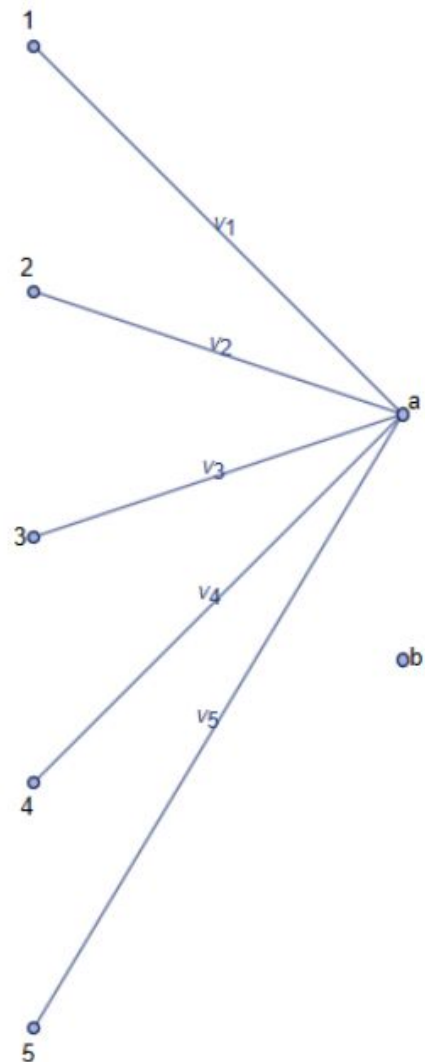


Picture credits: <https://home.cs.colorado.edu/~DrG/Courses/2026-Spring-NN/03-Fully-Connected-NN.pdf>
<https://imagecomics.com/comics/series/transformers>

What is possible



What we have



Do we have the right foundation for deep learning?

- **Recall:** backpropagation was questioned by its original authors
 - Not biologically plausible, alternatives exist as of 2022
 - Do we need to develop new learning algorithms?
- Despite the existence of many theories of DL, we still don't have a complete understanding of what, how, when, and why architectures generalize
 - Can we build theoretical tools to fill in these gaps?
- We've seen cases of unexpectedly good generalization performance arising from surprising design choices
 - Perhaps we're missing some pieces of the puzzle?

Questions to Ponder

- Will deep neural networks ever match the human brain's power efficiency?
 - We only need ~20 watts to perform complex tasks
- Is hyper-scaling the only way forward?
- Are there new types of architectures waiting to be discovered?
 - My (highly biased) answer is “yes” — finding them isn't so easy...
- Will we ever have a complete theory of generalization in DNNs?
- Many, many more...

Thank You!